

Pearl AQI Engine: Project Report

1. Project Overview

For my final project in the 10Pearls Shine Internship Program, I built the Pearl AQI Engine, an end-to-end, automated Air Quality Index (AQI) forecasting app for Islamabad.

The goal of this system is to pull real-time weather and pollution data, store it securely in a cloud database, automatically retrain a machine learning model every single day, and serve a 3-day (72-hour) forecast to an interactive web dashboard.

The Core Tech Stack

- **Data Storage:** OpenMeteo Air Quality API & Hopsworks Feature Store.
- **Machine Learning:** Scikit-learn (Random Forest) & MLflow for tracking.
- **Backend API:** FastAPI hosted on Render (with SHAP for explainability).
- **Frontend Dashboard:** React.js and Recharts.
- **Automation:** GitHub Actions (Running scripts on a schedule).

2. Data Collection & Feature Engineering

Instead of manually downloading datasets, I built a Python script that automatically gathers hourly pollution records.

Fetching the Data

My data script targets Islamabad. Every time it runs, it calls the OpenMeteo API to extract the last 90 days of atmospheric data. I specifically focused on:

- European AQI (The main target we are trying to predict)
- PM10 and PM2.5 (Particulate matter in the air)
- Nitrogen Dioxide

Making the Data "Smart"

Raw numbers aren't enough for an AI model to really understand the weather, so I added some custom columns (features) to the dataset before saving it:

- **Time Hints:** I extracted the hour, day_of_week, and month. This helps the model realize that pollution might spike during morning rush hour or get worse in winter months.

- **AQI Momentum:** I calculated if the air quality was actively rising or falling compared to the previous hour. This tells the model if the air is currently clearing out or getting more polluted.

3. The Brain: Machine Learning & MLflow

To build the actual prediction brain, I set up local experiment tracking using MLflow to test several different machine learning algorithms before deciding what to deploy.

Algorithm Selection & Engineering Trade-offs

During my notebook experiments, I tracked evaluation metrics (R^2 , RMSE, MAE) for five different setups: a Baseline Ridge model, a Linear Lasso model, a basic Decision Tree, a Random Forest, and an Ensemble Gradient Boosting model.

The linear models (Ridge and Lasso) performed terribly, scoring an R^2 of only about 0.05. This successfully proved my theory that the relationship between weather and pollution is highly complex and non-linear.

The tree-based ensemble models completely crushed the linear ones. Interestingly, the Gradient Boosting model technically achieved the highest raw performance score on the MLflow leaderboard (R^2 of ~ 0.457). However, I actively chose to deploy the Random Forest (R^2 of ~ 0.409) as the final production model instead.

Here is why I made that specific engineering trade-off:

- **Unsupervised Pipeline Stability:** Gradient Boosting builds trees sequentially, which makes it highly sensitive to hyperparameter drift over time. Random Forest builds independent trees in parallel, making it significantly more robust and stable for an unsupervised GitHub Actions robot that trains entirely on its own every day.
- **Compute Efficiency:** Because Random Forest trains in parallel, it consumes much less compute time. This is critical for keeping the pipeline running smoothly on the free-tier GitHub Actions runners.

The Model Registry

Every time the daily automation robot trains a new Random Forest, it gets evaluated and assigned a new R^2 score. I used Hopsworks as a Model Registry. My code checks the new model's score, and if it's the highest score we've ever seen, it saves the model to the cloud. This acts as an automated guardrail, protecting the live website from ever using a degraded model.

4. The Application Layer (FastAPI & React)

To actually show this data to users, I had to build a bridge between the cloud database and a website.

The Backend (FastAPI)

I wrote a server using FastAPI in Python. When it turns on, it downloads the absolute best version of the Random Forest model from Hopsworks. When a user visits the site, the API calculates a 72-hour forecast.

I also integrated SHAP (SHapley Additive exPlanations). Instead of just giving a final AQI number, SHAP looks inside the Random Forest and calculates exactly why the air is bad today (for example, it tells us if PM2.5 is the main reason for the pollution).

The Frontend (React)

I built a clean dashboard using React. It automatically grabs the forecast from the FastAPI server.

- It uses Recharts to draw a bar chart showing the SHAP drivers (what is causing the pollution).
- It draws a smooth line chart showing the 72-hour future horizon.
- I also added a dynamic function that changes the text on the screen depending on the air quality (e.g., if the air is bad, it says, "The air looks like soup. Consider breathing 20% less today.").

5. Automation (Building the Robot)

To make the system completely hands-free, I used GitHub Actions to act as the pipeline orchestrator.

- **The Hourly Data Pull:** A GitHub robot wakes up at minute 27 of every hour, runs the data extraction script, pushes the latest weather to the Hopsworks database, and goes back to sleep.
- **The Daily Training:** Another robot wakes up every day at 2:30 AM. It fetches all the data, trains a brand-new Random Forest model to see if it can learn new patterns, and saves it to the registry.

6. Issues Faced & What I Learned

This project was a massive learning curve. Encountering and resolving infrastructure bugs taught me more about real-world software engineering than the initial coding phase.

- **Hopworks Server Crashes:** Sometimes the free-tier cloud server was too busy and crashed my data downloads.

Fix: I added a retry loop in my code so it tries to connect 5 times before giving up.

- **Library Version Conflicts:** GitHub Actions kept downloading the absolute newest version of NumPy, which completely broke my Python environment.

Fix: I learned how to "pin" my library versions (like `numpy<2.0.0`) so the automation robot only uses stable, tested versions.

- **Accidental Time Travel (Data Leakage):** When splitting data for training, the code was shuffling the rows randomly. The model was basically cheating by looking at tomorrow's weather to guess yesterday's.

Fix: I turned off shuffling (`shuffle=False`) so the model trains chronologically.

- **Columns Shuffling in the Database:** The cloud database sometimes returned my data columns in a random order, which confused the ML model.

Fix: I hardcoded a strict list of columns (`FEATURE_COLS`) in my backend API so the data is always forced into the exact shape the model expects.

- **Predicting the Future Without Data:** To guess tomorrow's AQI, the model needs tomorrow's pollution levels, which obviously haven't happened yet.

Fix: I used a rule called the "Persistence Assumption." In my backend code, I froze today's pollution levels, but rolled the clock forward 3 days. This lets the AI calculate the daily ups and downs based on the time of day.

- **Strict Database Rules:** I tried adding a new timestamp column to my Python script, but Hopworks rejected the whole upload because the schema didn't match.

Fix: I learned that Feature Stores are super strict to prevent data corruption. I reverted my code to match the exact original database columns.

- **The 11:00 PM API Quirk:** Our "Current AQI" on the React site sometimes showed data from 11:00 PM (23:00 UTC) even if it was midday.

Fix: I realized OpenMeteo gives historical data in full 24-hour chunks. Instead of writing complex filtering code that might break, I used the latest available row from the database as my starting anchor point.

- **Timezone Math Breaking React:** The React site crashed with an "Invalid Date" error because of how UTC time formats use colons. Also, converting UTC to Pakistan Standard Time pushed the forecast into a 4th day.

Fix: I wrote a clean JavaScript function to strip the timezone offsets, and I sliced the array to exactly 3 days so it fits the UI perfectly.

7. AI Usage Statement

During this internship project, I used Large Language Models (LLMs) strictly as a pair-programming assistant. I used it for peer-reviewing my Python logic, checking for JavaScript syntax errors, and understanding complex MLOps concepts like Feature Stores and CI/CD pipelines. All architecture decisions, bug implementations, and final code structures were completed by me.

Project By: Ayaan Ahmed Khan